

Retail Fraud Detection: Machine Learning

Peiling Cheng
University of California San Diego
United States
pec021@ucsd.edu

Introduction

As technology advances, digital transactions have become more common. This brings convenience to everyone, including scammers, leading to a rise in fraud. Fraud detection systems attempt to identify suspicious transactions. However, identifying transactions as fraudulent when they are actually legitimate would disrupt customers. In this project, we aim to build a model to predict whether a transaction is fraudulent while avoiding false alarms. False positives disrupt customer behavior while false negatives lead to financial losses. Finding a balance between these is the ultimate goal of fraud detection.

1 Dataset

1.1 Data Source & Description

The dataset used in this project is the Retail Intelligence: Fraud Detection Dataset from Kaggle created by Noopur Bhatt. It is designed for fraud detection, anomaly detection, and risk scoring in retail transactions. This dataset combines transaction data, context signals, behavioral patterns, risk indicators, and the fraud label. We downloaded the dataset as a CSV file (retail_fraud_detection_100k.csv) and it contains 100,000 rows and 21 columns, taking up 11.86 MB of storage.

1.2 Exploratory Data Analysis

Basic Statistics & Properties

Each row represents a unique retail transaction, totaling 100,000 transactions with no missing values. It covers a one-year period from April 30, 2025 to April 30, 2026. There are 2 float columns, 11 integer columns, and 8 object columns representing categorical features.

Below is a description of each column:

- `transaction_id`: unique identifier for each transaction (object)
- `customer_id`: unique identifier for each customer (object)
- `transaction_timestamp`: date and time of the transaction (object)
- `transaction_amount`: dollar amount of the transaction (float64)
- `payment_method`: method used for payment (object)
- `device_type`: type of device used for the transaction (object)
- `location`: country where the transaction occurred (object)
- `merchant_category`: type of merchant (object)
- `is_international`: whether the transaction was international (int64)
- `transaction_frequency_24h`: number of transactions made by the customer in the last 24 hours (int64)

- `avg_transaction_amount_7d`: customer's average transaction amount in the last 7 days (float64)
- `failed_transaction_count_24h`: number of failed payment attempts by the customer in the last 24 hours (int64)
- `account_age_days`: number of days since the account was created (int64)
- `previous_fraud_flag`: whether the customer had previously been flagged with transaction fraud (int64)
- `unusual_amount_flag`: whether the transaction amount is unusual (int64)
- `unusual_location_flag`: whether the transaction location is unusual (int64)
- `multiple_transactions_short_time`: whether multiple transactions were made in a short period (int64)
- `high_risk_device_flag`: whether the transaction was made on a high risk device (int64)
- `velocity_flag`: whether the transaction velocity is high (int64)
- `fraud_flag`: whether the transaction was flagged as fraudulent (int64)
- `fraud_risk`: the level of risk (object)

Findings

Class Balance: The dataset has 47,508 (47.508%) transactions marked as fraud and 52,492 (52.492%) transactions marked as not fraud. This is a nearly 50/50 split which is not realistic for real-world data where fraudulent transactions are a much smaller percentage of retail transactions.

Transaction Amount: Transaction amounts range from \$0.00 to \$1,548.52. Fraudulent transactions have a higher average amount and a higher standard deviation (\$146.43, std = \$136.10) compared to legitimate transactions (\$96.99, std = \$97.01). This is 51% higher than the legitimate transactions average. In Figure 1, it is clear that there are more fraudulent transactions as the transaction amount increases. Looking further into the transaction amounts, we discovered that there were 4 transactions with an amount of zero, which is quite suspicious. However, 2 of the transactions were fraudulent and 2 were legitimate, so this would not affect our model.

Fraud Risk: The `fraud_risk` column is either low, medium, or high. However, we noticed that none of the "high" risks were legitimate and none of the "low" risks were fraudulent. This indicates potential target leakage.

Repeat Customers: As previously mentioned, there are 100,000 unique transaction IDs. However, there are only 28,955 unique customer IDs, indicating that many customers made multiple purchases. Two customer IDs appeared up to 13 times in our dataset, which we used to examine the `previous_fraud_flag` column.

Temporal Inconsistency: Looking at the transactions of customer C28986 (one of the two customers that had 13 transactions) and

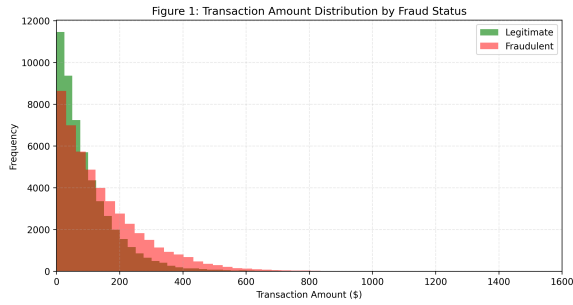


Figure 1: Overlapping histogram of transaction amounts by fraud status

sorting them by `transaction_timestamp`, we did not see a transaction marked with `previous_fraud_flag = 1` when the previous transaction was marked as fraud. This suggests that the `previous_fraud_flag` may be derived from an external source rather than from transactions within this dataset.

Transaction Timestamp: All transactions in the dataset occurred at 19:00 (7 PM). This is not realistic for real-world data where transactions can happen at any time. Therefore, the timestamp provides no predictive value and was excluded from modeling.

International Fraud Risk: The relationship between international status and fraud is shown in Figure 2. Among domestic transac-

	<code>fraud_flag</code>	Fraud	Not Fraud
<code>is_international</code>			
Domestic		26.5%	73.5%
International		68.7%	31.3%

Figure 2: Fraud rate by international status

tions, 73.5% were legitimate and 26.5% were fraudulent. Among international transactions, 31.3% were legitimate and 68.7% were fraudulent. International transactions are nearly 2.6 times more likely to be fraudulent.

Previous Fraud Flag Risk: The relationship between previous fraud history and fraud is shown in Figure 3. Among transactions with no previous fraud flag, 73.7% were legitimate and 26.3% were fraudulent. Among transactions with a previous fraud flag, 31.3% were legitimate and 68.7% were fraudulent. Transactions with a previous fraud flag are 2.6 times more likely to be fraudulent.

Feature Independence: Given that both international status and previous fraud flag show a $\sim 2.6x$ increased risk, we examined their relationship. The distribution was nearly even, with each combination near 25%, indicating the two features are independent. With no multicollinearity, both features can be used in our modeling.

	<code>fraud_flag</code>	Fraud	Not Fraud
<code>previous_fraud_flag</code>			
No Previous Fraud		26.3%	73.7%
Previous Fraud		68.7%	31.3%

Figure 3: Fraud rate by previous fraud flag

2 Predictive Task

2.1 Problem Definition

The task is to predict whether a transaction is fraudulent or not. This is a binary classification problem where `fraud_flag` equals 1 if the transaction is fraud and 0 if it is legitimate.

2.2 Evaluation Metrics

We used two metrics to evaluate our models: accuracy and F1 score. Accuracy measures the percentage of correctly predicted transactions. It is calculated as:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

where TP = true positives, TN = true negatives, FP = false positives, and FN = false negatives.

F1 score is the harmonic mean of precision and recall. It is calculated as:

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

where Precision = $TP / (TP + FP)$ and Recall = $TP / (TP + FN)$. Accuracy alone can be very misleading in fraud detection. For example, if only 5% of a dataset is fraudulent, then a model guessing not fraudulent all the time would get an accuracy of 95%. This would mean guessing not fraudulent for all the fraudulent transactions (false negatives), leading to merchants losing money. F1 scores address this problem by penalizing false negatives and false positives. Hence, the F1 score is more informative than accuracy alone since it avoids making those mistakes.

2.3 Baselines

Most features in this dataset are numerical. Using all numerical features would make the baseline model too robust so we selected the top three correlated features (see Figure 4): `is_international`, `previous_fraud_flag`, and `unusual_location_flag`. Each of these features showed a correlation of approximately 0.42 with `fraud_flag`. A random state of 42 was used for reproducibility and to ensure a fair comparison between the models. We performed an 80/20 train-test split. Default hyperparameters were used.

Dummy Classifier always predicts the most frequent class (legitimate) and establishes the minimum performance floor.

Naïve Bayes assumes feature independence, which matches our findings from EDA.

Logistic Regression assumes a linear relationship between features and fraud. It works well with binary features, which the

fraud_flag	1.000000
previous_fraud_flag	0.423561
is_international	0.422911
unusual_location_flag	0.422911
multiple_transactions_short_time	0.348364
unusual_amount_flag	0.329398
velocity_flag	0.326297
transaction_frequency_24h	0.324047
failed_transaction_count_24h	0.228550
transaction_amount	0.206105
high_risk_device_flag	0.126393
account_age_days	-0.002136
avg_transaction_amount_7d	-0.138906

Figure 4: Feature correlations with fraud_flag

majority of the columns are. StandardScaler was applied to prevent large-range features (e.g., transaction amounts) from taking on too much weight.

k-NN (k=5) makes predictions based on the majority class of its five closest neighbors. It is a distance-based classifier that captures local patterns. StandardScaler was applied to avoid large-range features from taking on too much weight.

2.4 Baseline Results

The baseline models were evaluated on the 20% test set using accuracy and F1 scores. Results are summarized in Figure 5. Logistic re-

Model	Accuracy	F1 Score
Dummy Classifier (majority)	0.5181	0.0000
Naïve Bayes	0.7076	0.7016
Logistic Regression	0.7329	0.6363
k-NN (k = 5)	0.7076	0.7016

Figure 5: Baseline model performance comparison

gression achieved the highest accuracy but a lower F1 score, which is important for fraud classification. Naïve Bayes and k-NN tied with the same accuracy and F1 score. Both models have a slightly lower accuracy but a higher F1 score than logistic regression. We selected Naïve Bayes and k-NN for optimization despite the two models getting the same scores because they are fundamentally different models. Naïve Bayes uses a probabilistic approach and k-NN uses a distance-based approach.

3 Model

3.1 Initial Settings

Dataset: Retail Intelligence: Fraud Detection Dataset.

Initial features: same as the baseline (top three correlated features: is_international, previous_fraud_flag, unusual_location_flag) We optimized Naïve Bayes and k-NN. Naïve Bayes performs well with independent features and k-NN makes it easy to observe local patterns. Optimization is based on improving accuracy and F1

scores. We explored additional features that increase these two metrics.

3.2 Feature Engineering & Results

We added features incrementally based on their correlation with fraud_flag and tracked accuracy and F1 score for each new feature (see Figure 6). Next, we experimented with one-hot encoding for the

Naïve Bayes:			
Features	Last added	Accuracy	F1 Score
3	unusual_location_flag	0.7076	0.7016
4	multiple_transactions_short_time	0.7076	0.7016
5	unusual_amount_flag	0.8673	0.8611
6	velocity_flag	0.9049	0.9060 <<< BEST
7	transaction_frequency_24h	0.8639	0.8620
8	failed_transaction_count_24h	0.8880	0.8863
9	transaction_amount	0.8717	0.8664
10	avg_transaction_amount_7d	0.8749	0.8694
11	high_risk_device_flag	0.8681	0.8609
12	account_age_days	0.8681	0.8608
k-NN:			
Features	Last added	Accuracy	F1 Score
3	unusual_location_flag	0.7076	0.7016
4	multiple_transactions_short_time	0.8470	0.8385
5	unusual_amount_flag	0.8547	0.8435
6	velocity_flag	0.9049	0.9060
7	transaction_frequency_24h	0.9028	0.9031
8	failed_transaction_count_24h	0.9773	0.9760
9	transaction_amount	0.9778	0.9764
10	avg_transaction_amount_7d	0.9774	0.9761
11	high_risk_device_flag	0.9999	0.9999 <<< BEST
12	account_age_days	0.9991	0.9991

Figure 6: Model performance by number of features added (Naïve Bayes and k-NN)

four categorical features (excluding fraud_risk): payment_method, device_type, location, and merchant_category. Each categorical feature was added separately to evaluate whether it would improve model performance (see Figure 7). OneHotEncoder from

```

=====
Adding Categorical Features One by One (k-NN)
=====

Baseline (numerical only): F1 = 0.9999
Baseline + device_type: F1 = 0.9992
Baseline + merchant_category: F1 = 0.9980
Baseline + payment_method: F1 = 0.9984
Baseline + location: F1 = 0.9976

=====
Adding Categorical Features One by One (Naïve Bayes)
=====

Baseline (numerical only): F1 = 0.9060
Baseline + device_type: F1 = 0.9060
Baseline + merchant_category: F1 = 0.9060
Baseline + payment_method: F1 = 0.9060
Baseline + location: F1 = 0.9060

```

Figure 7: Effect of one-hot encoded categorical features on model performance

sklearn.preprocessing was used and StandardScaler was applied for k-NN.

For Naïve Bayes, the best performance was achieved with 6 features, with an accuracy of 0.9049 and an F1 score of 0.9060. Adding features beyond that decreased performance.

For k-NN, the best performance was achieved with 11 features, with an accuracy of 0.9999 and an F1 score of 0.9999.

3.3 Optimization & Problems

Given that both models achieved near-perfect performance with default settings, hyperparameter tuning was not necessary. We selected k-NN as the final model since it achieved the highest F1 score (0.9999).

We did not encounter any scalability or overfitting issues because the dataset was balanced and did not have complex patterns. However, real-world fraud datasets are not this "perfect," so our results would not directly reflect real-world patterns.

4 Literature

Fraud detection is a well-established research problem across different domains (e.g., medical billing, insurance claims, and credit card transactions). Researchers typically use supervised learning with ensemble methods to achieve strong performance. Our work differs by using a balanced dataset, which eliminates the need for resampling techniques like Synthetic Minority Over-sampling Technique (SMOTE) and allows simpler models to perform well. Ensemble methods such as Random Forest and XGBoost are commonly used because they effectively handle non-linear patterns and class imbalance. However, there is a tradeoff between effectiveness and computational cost.

Comparative Survey of AI-Driven Credit Card Fraud Detection

<https://ieeexplore.ieee.org/document/11298125>

This survey focuses on credit card fraud and compares three categories of methods: machine learning, deep learning, and hybrid. Machine learning methods reviewed include decision trees, support vector machines, random forests, logistic regression, KNN, and linear discriminant analysis. KNN achieved an accuracy of 94.35%-95.70%, and it also performed strongly in our results. Deep learning methods reviewed include CNN, RNN, GNN, LSTM, ANN, and transformer. Although deep learning methods are better for complex, non-linear patterns, they are computationally expensive. Hybrid strategies combine the advantages of machine learning and deep learning. Out of these, machine learning combined with deep learning can achieve accuracy up to 99.99%. Metrics used in the survey were accuracy, F1 score, precision, AUC, and recall. Our results for KNN are consistent with the results mentioned in this survey.

Credit Card Fraud Detection in Banking Using Machine Learning

<https://ieeexplore.ieee.org/document/11048184>

This study focuses on credit card fraud using machine learning techniques such as feature engineering, anomaly detection, and ensemble learning. The authors obtained their dataset from Kaggle with 4,850 records and 11 fields, which is much smaller than our

100,000 row dataset. Because their dataset was not balanced, the authors used SMOTE. Similar to our dataset, each record represented one transaction. Models tested include decision trees, random forest, and support vector machines. Decision tree and random forest classifiers achieved a 1.0 on accuracy, precision, recall, and F1. Their KNN model achieved an accuracy of 0.996 and an F1 score of 0.996, similar to our results. Their Naïve Bayes model performed poorly with an accuracy of 0.335 and an F1 score of 0.31. This is much lower than our Naïve Bayes model, likely due to feature independence violations, class imbalance, and their smaller dataset size.

5 Results

5.1 Comparison

After feature engineering, our Naïve Bayes model achieved its best performance with the top 6 correlated features. It was able to achieve an accuracy of 0.9049 and an F1 score of 0.9060. Our k-NN model achieved its best performance with the top 11 correlated features, reaching an accuracy of 0.9999 and an F1 score of 0.9999. This represents a significant improvement from their baseline performance, with an increase in accuracy and F1 score of at least 0.20.

5.2 Effectiveness

For both models, accuracy and F1 score decreased after the optimal number of features. One-hot encoded categorical features either did not improve or hurt performance. For example, adding device_type to k-NN decreased F1 from 0.9999 to 0.9992. Since individual categorical features did not improve performance, combinations of these features were not explored. Therefore, we used the optimal feature set identified for each model (top 6 for Naïve Bayes and top 11 for k-NN).

5.3 Hyperparameter

Naïve Bayes used default settings and k-NN used default settings with $k = 5$. Both models achieved near-perfect performance with default settings, so hyperparameter tuning was unnecessary.

5.4 Takeaways

- Balanced datasets allow simple models to achieve strong performance. Our balanced dataset eliminated the need for resampling techniques like SMOTE.
- Feature selection by correlation is effective. Adding features in order of their correlation helped us identify the optimal feature set for each model and avoid hurting performance. Adding features beyond this point decreased performance.
- Features that intuitively seem important can actually hurt performance. One-hot encoded categorical features seemed relevant but ended up having no effect or hurting performance.
- The optimal number of features differs by model. Naïve Bayes achieved its best performance with only 6 features, while k-NN required 11 features to reach near-perfect performance.

References

- [1] <https://www.kaggle.com/datasets/noopurbhatt/retail-intelligence-fraud-detection-dataset>
- [2] <https://ieeexplore.ieee.org/document/11298125>
- [3] <https://ieeexplore.ieee.org/document/11048184>
- [4] <https://scikit-learn.org/stable/>
- [5] <https://pandas.pydata.org/>
- [6] <https://numpy.org/>
- [7] <https://matplotlib.org/>
- [8] <https://seaborn.pydata.org/>